CHƯƠNG 5
TẦNG ỨNG DỤNG

Bản tóm tắt để đọc nhanh và ôn thi
Nguồn chính: slide-vi 05 - Application Layer
Bổ sung: slide-en Chapter 2 (HTTP, email, P2P, video, socket)

1 Khái niệm cơ bản	2 Kiến trúc ứng dụng	3 Yêu cầu và dịch vụ transport
4 Lập trình socket TCP	5 Lập trình socket UDP	★ Ôn tập và câu hỏi

Mục tiêu bài học

- Phân biệt process, kiến trúc client-server và peer-to-peer; hiểu "địa chỉ" của một process (IP + port).
- Chọn dịch vụ TCP hay UDP cho một ứng dụng dựa trên yêu cầu về tin cậy, thời gian, băng thông, bảo mật.
- Nắm trình tự các hàm socket của server và client trong TCP (4 giai đoạn) và UDP.

Chú thích màu: khối tím là nội dung bổ sung từ slide tiếng Anh. Chỗ ghi **[cần kiểm tra lại]** là điểm slide chưa rõ. Slide-vi mục "DHCP, DNS" tham chiếu sang chương 3.

1 Khái niệm cơ bản

- **Process** (tiến trình): chương trình đang chạy trên máy; có thể có nhiều thread (tiểu trình). Trên cùng một máy, các process liên lạc qua hệ điều hành (chia sẻ bộ nhớ hoặc truyền thông điệp); trên hai máy khác nhau thì truyền dữ liệu qua mạng (ví dụ socket, named pipe).
- **Ứng dụng mạng** chạy trên các end-system và liên lạc với nhau qua mạng.
- **"Địa chỉ" của process** = địa chỉ IP (xác định máy) + **port** (xác định process trên máy). Phân loại port: **0..1023** port chuẩn; **1024..49151** port cố định đăng ký trước; phần còn lại là port linh động.
- **Giao thức tầng ứng dụng** do người xây dựng ứng dụng định nghĩa (ví dụ HTTP, FTP).

BỔ SUNG TỪ SLIDE-EN · Giao thức tầng ứng dụng định nghĩa gì (slide-en 11-16)

- Loại thông điệp (request, response), cú pháp (các trường và cách phân tách), ngữ nghĩa (ý nghĩa các trường), và quy tắc khi nào/làm sao gửi và trả lời.
- Có thể là giao thức mở (như HTTP, SMTP - định nghĩa trong RFC) hoặc độc quyền (như Skype).
- Với tính bảo mật: TCP/UDP thuần không mã hóa; **TLS** chạy ở tầng ứng dụng, thêm mã hóa, toàn vẹn dữ liệu và xác thực đầu cuối.

2 Kiến trúc ứng dụng: client-server và P2P

	Client-server	Peer-to-peer (P2P)
Vai trò	Server luôn "sống", nhận và xử lý yêu cầu; client gửi yêu cầu	Mỗi máy vừa là server vừa là client
Địa chỉ	Server có địa chỉ cố định; client có thể dùng IP động	Dùng địa chỉ động
Liên lạc	Hai client không liên lạc trực tiếp với nhau	Các peer liên lạc trực tiếp
Quản lý	Dễ quản lý	Khó quản lý
Ví dụ	Web (Apache, IIS và trình duyệt), FTP	Skype, BitTorrent

BỔ SUNG TỪ SLIDE-EN · Thời gian phân phối file: client-server so với P2P (slide-en 75-78)

- Ký hiệu: **F** kích thước file; **N** số peer; $\mathbf{u_s}$ tốc độ upload của server; $\mathbf{u_i}$ tốc độ upload của peer i; $\mathbf{d_{min}}$ tốc độ download nhỏ nhất của các client.

$D_{c\text{-}s} \geq \max\{ N{\cdot}F/u_s , F/d_{min} \}$	$D_{P2P} \geq \max\{ F/u_s , F/d_{min} , N{\cdot}F/(u_s + \Sigma u_i) \}$
Client-server: tăng tuyến tính theo N	P2P: tự mở rộng vì mỗi peer thêm năng lực upload

- Slide-en ghi công thức bằng ký hiệu max{...}; dấu so sánh trong bản slide hiển thị là ">" ($D_{c\text{-}s}$) và ">" (D_{P2P}); ở đây viết ≥ cho đúng ý nghĩa cận dưới **[cần kiểm tra lại]**.
- **BitTorrent:** file chia thành chunk 256 Kb; **tracker** theo dõi các peer; **torrent** là nhóm peer trao đổi chunk. Peer mới có 0 chunk, đăng ký với tracker để lấy danh sách peer. **Churn**: peer đến và đi bất kỳ lúc nào.
- Xin chunk: chunk **hiếm nhất trước** (rarest first). Gửi chunk: **tit-for-tat** - gửi cho 4 peer đang gửi cho mình nhanh nhất (xét lại mỗi 10 s), các peer khác bị **choked**; mỗi 30 s chọn ngẫu nhiên một peer để "optimistically unchoke".

3 Yêu cầu của ứng dụng và dịch vụ transport

- **Yêu cầu** của tầng ứng dụng: truyền dữ liệu đáng tin cậy, thời gian (độ trễ), băng thông, bảo mật dữ liệu.
- **TCP**: dịch vụ hướng kết nối, truyền đáng tin cậy. **UDP**: hướng không kết nối, không đáng tin cậy, nhanh hơn TCP.
- Slide-vi có bảng port của các ứng dụng chạy trên TCP/UDP (slide 12, dạng hình ảnh) và dẫn tới danh sách của IANA **[cần kiểm tra lại]** các số port cụ thể.

Dịch vụ	TCP	UDP
Kết nối	Cần thiết lập kết nối giữa client và server	Không kết nối, không handshake
Độ tin cậy	Truyền đáng tin cậy	Có thể mất hoặc đến sai thứ tự
Tốc độ	Chậm hơn (có kiểm soát)	Nhanh hơn

BỔ SUNG TỪ SLIDE-EN · HTTP (slide-en 18-47)

- **Non-persistent HTTP**: mỗi object một kết nối TCP; thời gian ≈ **2·RTT + thời gian truyền file** cho mỗi object. **Persistent HTTP**: nhiều object qua một kết nối, server giữ kết nối mở, khoảng **1 RTT** cho mọi object tham chiếu.
- Thông điệp **request** (ASCII): dòng yêu cầu (method, URL, version), các dòng header, dòng trống, body. Method: GET, POST, HEAD, PUT. **Response**: dòng trạng thái và header, rồi dữ liệu.
- Mã trạng thái: **200** OK; **301** Moved Permanently; **400** Bad Request; **404** Not Found; **505** HTTP Version Not Supported.
- **Cookie**: server dùng để lưu trạng thái người dùng (HTTP vốn "không trạng thái") với 4 thành phần: dòng Set-cookie trong response, dòng Cookie trong request, file cookie trên máy người dùng, cơ sở dữ liệu ở server.
- **Web cache (proxy server)**: trả nội dung gần client, giảm thời gian đáp ứng và giảm tải đường ra ISP. Ví dụ slide với hit rate 0,4: utilization đường truy cập giảm từ 0,9 về khoảng 0,58 và độ trễ trung bình còn khoảng 1,2 s **[cần kiểm tra lại con số trên slide gốc]**.
- **Conditional GET**: client gửi If-modified-since; server trả **304 Not Modified** nếu bản cache còn mới, tránh gửi lại object.
- **HTTP/2** giảm head-of-line (HOL) blocking bằng cách chia object thành frame và xen kẽ gửi; **HTTP/3** chạy trên QUIC/UDP để thêm bảo mật và kiểm soát tắc nghẽn.

BỔ SUNG TỪ SLIDE-EN · E-mail: SMTP, IMAP (slide-en 49-57)

- Ba thành phần: **user agent**, **mail server** (hộp thư đến và hàng đợi gửi đi), và giao thức **SMTP** giữa các mail server.
- SMTP dùng **TCP, port 25**, truyền trực tiếp từ server gửi đến server nhận, 3 pha: bắt tay, truyền thông điệp, đóng. Lệnh tiêu biểu: HELO, MAIL FROM, RCPT TO, DATA, QUIT; kết thúc nội dung bằng dòng chỉ có dấu chấm.
- So với HTTP: HTTP là **pull** (client kéo), SMTP là **push** (client đẩy); SMTP cần thông điệp 7-bit ASCII; SMTP dùng kết nối persistent.
- Nhận thư dùng giao thức truy cập như **IMAP** (giữ thư và thư mục trên server) thay vì SMTP.

4 Lập trình socket với TCP

- **Socket**: "cánh cửa" giữa ứng dụng và giao thức tầng transport (TCP, UDP); là một end-point của liên kết giữa hai ứng dụng và là giao diện lập trình mạng ở tầng Transport. Windows có thư viện Winsock API.
- **5 việc cần xác định khi lập trình ứng dụng mạng:** (1) kiến trúc client-server hay P2P; (2) giao thức transport TCP hay UDP; (3) port dùng ở server và client; (4) giao thức tầng ứng dụng để trao đổi dữ liệu; (5) lập trình.

Giai đoạn	Server	Client
1. Tạo socket, lắng nghe	**socket()** → **bind()** (gắn với port) → **listen()**	-
2. Thiết lập kết nối	**accept()**: chờ và chấp nhận, tạo **socket mới** cho kết nối đó	**socket()** → **connect()** tới server
3. Trao đổi dữ liệu	**send()/receive()** trên socket của connection	**send()/receive()**
4. Kết thúc	**close()** socket của connection	**close()**

BỔ SUNG TỪ SLIDE-EN · Ví dụ Python TCP (slide-en 106-109)

- Server tạo socket lắng nghe ("cửa chào") bằng **socket(AF_INET, SOCK_STREAM)**, bind port 12000, **listen(1)**; trong vòng lặp **accept()** trả về **connectionSocket** mới để nhận câu, đổi thành chữ hoa rồi gửi lại và đóng.
- Client: **socket(AF_INET, SOCK_STREAM)**, **connect((serverName, 12000))**, **send** câu nhập vào, **recv(1024)** câu đã đổi chữ hoa rồi **close()**.
- Server dùng **socket mới cho mỗi client** nên phục vụ được nhiều client; phân biệt client bằng source port (chi tiết ở chương transport).

5 Lập trình socket với UDP

- Mô hình UDP có 3 giai đoạn: (1) server tạo socket tại port; (2) client tạo socket; (3) trao đổi thông tin. Các hình chi tiết trong slide-vi (slide 23-25) là hình ảnh, chỉ ghi lại ý chính ở đây **[cần kiểm tra lại]** với slide gốc.

BỔ SUNG TỪ SLIDE-EN · UDP không connection (slide-en 102-105)

- Không handshake; người gửi **tự gắn IP đích và port đích** vào mỗi gói, người nhận lấy IP và port của người gửi từ gói nhận được. Dữ liệu có thể mất hoặc sai thứ tự.
- Server: **socket(AF_INET, SOCK_DGRAM)**, **bind(('', 12000))**, rồi lặp **recvfrom(2048)** và **sendto(...)** về địa chỉ client. Client: tạo socket, **sendto** tới (serverName, 12000), **recvfrom(2048)**, **close()**. Không có listen/accept/connect.

★ Ôn tập nhanh

Cần thuộc

Port	TCP server	TCP client	UDP
0-1023 chuẩn; 1024-49151 đăng ký; còn lại động	socket, bind, listen, accept, send/recv, close	socket, connect, send/recv, close	sendto / recvfrom, không connect

Lỗi thường gặp

Dễ nhầm	Cần nhớ
Địa chỉ của process	IP + port, không chỉ IP
Socket của accept()	Tạo socket MỚI cho từng client; socket lắng nghe vẫn tiếp tục chờ
TCP và UDP	TCP: hướng kết nối, tin cậy; UDP: không kết nối, không tin cậy, nhanh
Client trong client-server	Hai client không nói chuyện trực tiếp; P2P thì có
HTTP non-persistent và persistent	Non-persistent: 2 RTT + truyền / object; persistent: kết nối mở nhiều object (slide-en)
SMTP và HTTP	SMTP push, HTTP pull; SMTP dùng port 25 (slide-en)

Câu hỏi ôn tập (che cột đáp án rồi tự trả lời)

Câu hỏi	Đáp án
1. Hai kiến trúc ứng dụng mạng chính?	Client-server và peer-to-peer
2. "Địa chỉ" của một process gồm gì?	Địa chỉ IP và số port
3. Dải port chuẩn?	0 đến 1023
4. Kể 4 yêu cầu dịch vụ của ứng dụng đối với transport.	Tin cậy, thời gian, băng thông, bảo mật
5. Thứ tự hàm của TCP server?	socket() → bind() → listen() → accept() → send/receive → close()
6. Hàm nào client TCP dùng để bắt đầu kết nối?	connect()
7. accept() trả về gì?	Một socket mới dành riêng cho kết nối với client đó
8. UDP có cần listen/accept/connect không?	Không; dùng sendto/recvfrom, mỗi gói tự mang địa chỉ đích
9. Công thức $D_{c\text{-}s}$ cho thấy điều gì?	Thời gian phân phối tăng tuyến tính theo N (slide-en)
10. BitTorrent chọn chunk xin trước theo nguyên tắc nào?	Rarest first; gửi theo tit-for-tat cho 4 peer nhanh nhất (slide-en)

Thực hành gợi ý

- Chạy thử ví dụ Python UDP và TCP (đổi chữ thường thành chữ hoa) trên hai cửa sổ terminal, rồi bắt gói bằng Wireshark để thấy bắt tay 3 bước chỉ có ở TCP.
- Dùng telnet kết nối port 80 của một web server và gõ tay một request GET để thấy dòng trạng thái 200 hoặc 404.